

6주차

소프트웨어 시스템 설계 및 개발

2024.1학기

CONTENTS

1. HTTP 메서드
2. REST

HTTP 메서드

• HTTP 메서드

- 클라이언트가 서버로 보내는 HTTP 요청할 때, 요청의 성격이 무엇인지 알리기 위해 HTTP 메서드를 같이 보냄
- 주요 메서드로는 GET, POST 등이 있음

• HTTP 응답코드

- 서버는 클라이언트의 서버 요청을 수행한 후 그 결과를 응답코드와 함께 보냄
- 대표적인 응답코드는 200(성공), 400(잘못된 요청), 404(페이지 없음), 500(내부 서버 오류) 등이 있음

메서드	설명
GET	URL(URI) 형식으로 서버에 요청
POST	HTTP의 Body에 내용을 담아서 새로운 정보 생성을 서버에 요청
HEAD	GET과 비슷하며 실제 문서가 아닌 문서 정보(Header)만 요청
OPTIONS	서버가 지원하는 HTTP 메서드에 대한 질의
PUT	POST와 비슷하나 새로운 정보가 아닌 주로 기존 정보를 업데이트 요청
DELETE	정보의 제거 요청
TRACE	요청 리소스가 수신되는 경로를 보임
CONNECT	프락시 서버 같이 중간 서버 경유 요청

- HTTP 응답코드(1/2)

종류	내용
100번대	(정보) 요청을 받았으며 프로세스를 계속함
200번대	(성공) 요청을 받았고 수행하여 완료함
300번대	(리다이렉션) 요청 완료를 위해 추가적으로 작업이 필요함
400번대	(클라이언트 오류) 요청이 잘못 되거나 처리 불가
500번대	(서버 오류) 서버에 문제가 생겨서 처리 불가

• HTTP 응답코드(2/2)

응답 코드	의미	설명
200	OK	요청이 처리됨
201	Created	서버가 새 리소스를 작성함
204	No Content	요청은 성공했으나 컨텐츠가 없음
301	Moved Permanently	요청한 페이지는 영구히 이동하였음
302	Moved Temporarily	요청한 페이지는 임시로 이동하였음
400	Bad Request	서버가 요청을 인식 못 함(잘못된 요청)
401	Unauthorized	인증이 필요한 요청
403	Forbidden	서버가 요청을 금지함
404	Not Found	서버가 요청한 페이지를 찾을 수 없음
500	Internal Server Error	서버에 오류가 발생하여 요청 수행 불가
503	Service Unavailable	서버 과부하 등의 이유로 서버를 이용할 수 없음

실습1

- 브라우저로부터 사용자 입력을 받아서 서버에서 파일로 저장
- **week6_ex/ex1** 참고

```
project/  
├── server.js  
├── index.html  
└── students.txt
```

- JSON

- JavaScript Object Notation은 데이터 교환을 위해 개발된 경량 데이터 형식
- JSON은 언어 독립적이며, 대부분의 프로그래밍 언어에서 사용이 가능함
- 웹 개발에서 클라이언트<->서버 간의 데이터를 전송하는 표준 방식으로 많이 사용됨
- {}로 감싸져 있으면 k,v 쌍이 나와야 함. 순서는 상관없음
- []는 배열로 어떤 형태라도 들어갈 수 있으며 순서가 있음

```
{
  "name": "Park Jonggyu",
  "age": 38,
  "isStudent": false,
  "address": {
    "street": "Chungryeol-ro",
    "city": "Busan"
  },
  "phoneNumbers": ["010-1234-5678", "070-0000-9999"]
}
```

```
// 객체를 JSON 문자열로 변환
const jsonObj = { name: "Jong", age: 38 };
const jsonString = JSON.stringify(jsonObj);
console.log(jsonString);

// JSON 형태의 문자열을 객체로 변환
const jsonString = '{"name":"Jong","age":38}';
const jsonObj = JSON.parse(jsonString);
console.log(jsonObj.name);
```

실습2

- 브라우저로부터 사용자 입력을 받아서 서버에서 json 파일로 저장
- list 페이지로 이동하면 json 파일의 내용을 출력
- week6_ex/ex2 참고

```
project/  
├── server.js  
├── index.html  
└── students.json
```

- **REST(Representational State Transfer)**

- **REST**는 웹을 위한 네트워크 아키텍처 스타일로 아래와 같은 특성을 가짐
- **RESTful** 하다는 것은 REST 아키텍처 원칙을 준수한다는 것

특성	내용
Client-Server Architecture	클라이언트와 서버는 서로 독립적으로 작동해야 하며, 이를 통해 각각의 부분을 독립적으로 발전시킬 수 있음
Stateless	각 요청은 독립적이어야 하며, 서버는 클라이언트의 상태 정보를 저장하지 않아야 함. 모든 필요한 정보는 요청과 함께 전송되어야 함
Cacheable	클라이언트는 응답을 캐시할 수 있어야 하며, 서버 응답은 캐싱 가능 여부를 명시해야 함
Uniform Interface	인터페이스의 통일성을 통해 시스템의 각 부분을 독립적으로 개선할 수 있음. 이를 위해 리소스 식별, 자기 서술적 메시지, 하이퍼미디어 등의 원칙을 따름
Layered System	클라이언트는 서버가 직접적으로 연결되어 있는지, 중간 서버를 통해 연결되어 있는지 알 수 없어야 함
Code on Demand (Optional)	서버가 클라이언트에 실행 가능한 코드를 전송할 수 있어야 함(옵션)

REST (Stateless)

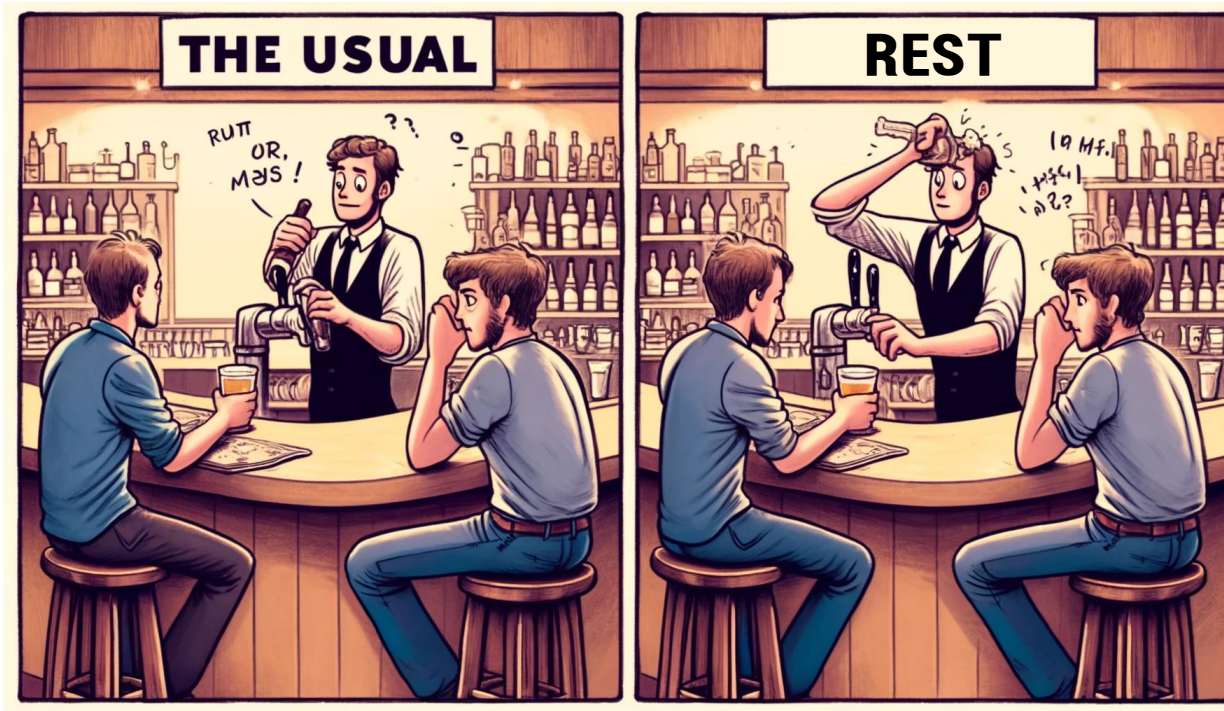
2.REST

- Stateless

- 클라이언트와 서버 간의 각 통신은 완전히 독립적이어야 하며, 서버는 클라이언트의 상태나 세션 정보를 저장하면 안 됨
- 대신, 각 요청은 필요한 모든 정보를 포함해야 하고, 서버는 그 요청만을 처리하여 응답해야 함
- 요청이 서로 의존적이지 않기 때문에, 각 요청은 이전의 다른 요청들로부터 독립적

능 마시던 걸로...

네! 능 마시던
봄베이로!



능 마시던 걸로...

그게 뭔데...?

- **State를 저장 안 함**

- **독립성**

- 각 요청이 필요한 모든 정보를 포함하므로, 서버는 이전 요청의 상태를 기억할 필요가 없어서 서버 설계를 단순화하고, 다른 서버로 요청을 자유롭게 전달할 수 있게 함

- **확장성**

- 서버가 상태 정보를 유지하지 않기 때문에, 어떤 서버에 요청이 배치되더라도 모든 서버가 동일하게 요청을 처리할 수 있음
 - 이는 서버 클러스터의 확장성을 증가시키며, 서비스의 가용성과 장애 허용성을 향상시킴

- **성능**

- 각 요청이 독립적으로 처리되므로, 서버는 상태를 관리하기 위한 추가적인 리소스를 사용할 필요가 없어 서버의 처리 성능을 최적화함

- **State를 저장함**

- **복잡성 증가**

- 서버가 클라이언트의 상태를 추적하려면 추가적인 로직과 저장 공간이 필요하여, 서버의 복잡성을 증가시키고, 오류 발생 가능성을 높임

- **확장성 제한**

- 서버가 상태 정보를 저장하면, 특정 클라이언트의 요청을 처리할 수 있는 서버가 제한될 수 있어 로드 밸런싱을 어렵게 만들고, 서버 확장을 복잡하게 할 수 있음

- **리소스 사용 증가**

- 상태 정보를 저장하고 관리하기 위해 추가 서버 리소스가 필요하므로 전체 시스템의 성능에 부담을 줌

세션과 쿠키

• 세션(Session)

- 세션은 **서버 측에서 사용자 데이터를 저장**
- 세션 ID는 일반적으로 쿠키를 통해 클라이언트에 저장되며, 이 ID를 사용하여 서버는 세션 데이터에 접근하고 사용자를 식별
- 세션은 주로 로그인 상태 유지에 사용

• 쿠키(Cookies)

- **클라이언트 측에서 작은 데이터 조각을 저장**
- 이 데이터는 웹 브라우저에 저장되며, 사용자가 웹사이트를 다시 방문할 때마다 웹 서버로 전송
- 쿠키는 사용자의 선호, 세션 트래킹, 사용자 식별 정보 등을 저장

• 그럼 세션과 쿠키를 쓰면 RESTful 하지 않은가...?

- 그렇다
- 원칙적으로는 세션과 쿠키는 REST 아키텍처를 위반하는 것이지만, 사용자 편의를 위해 실제 서비스에서는 세션과 쿠키를 사용하고 있음
- 따라서 세션과 쿠키를 쓰더라도 RESTful 하다고 표현하는 경우가 많음



REST API

- REST API는 REST 아키텍처를 기반으로 한 API(Application Programming Interface)
- REST API의 구성요소
 - **GET** : 리소스를 조회
 - **POST** : 새 리소스를 생성
 - **PUT** : 리소스를 갱신
 - **DELETE** : 리소스를 삭제
- REST API 사용의 장점
 - **간단함과 범용성**: HTTP 프로토콜을 사용하므로, REST API는 이해하기 쉽고 사용하기 간편함
 - **언어와 플랫폼 독립적**: 어떤 프로그래밍 언어나 플랫폼에서도 사용할 수 있음
 - **확장성과 유연성**: 새로운 리소스나 메서드를 기존 시스템에 쉽게 추가할 수 있음

- REST API로 구현한 CRUD 실행([week6_ex/ex3 참고](#)))
- POSTMAN 실습 시연

동작	메서드	url (contents)
사용자 전체 조회	GET	/users
특정 사용자 조회	GET	/user/1
사용자 추가	POST	/user Body:raw, JSON {"name":"Choi"}
사용자 수정	PUT	/user/1 Body:raw, JSON {"name":"Han"}
사용자 삭제	DELETE	/user/1

실습4

- 프론트엔드 서버에서 REST API 서버의 조회 기능 활용 (week6_ex/ex4 참고))
- ex3의 rest_api 서버 실행 (3000포트)
- ex4의 frontend_server 서버 실행 (3001포트)

실습5

- 실습2를 개선하여 json 파일 기반의 단순한 회원가입/로그인 서비스를 구현해보세요.
- 실습3을 활용하여 회원가입/로그인 기능을 rest api로 구현해보세요.